



LFCS

Linux Foundation Certified Sysadmin

Study Notes & Reference Guide

Complete Linux Sysadmin Preparation Notes

2025 EDITION

10

Sections

40+

Topics

400+

Commands

Author: Akhilesh Chaudhari

Linux Foundation Certification Preparation Guide

LFCS Study Notes | 2025 Edition

TABLE OF CONTENTS

LFCS Study Notes — 10 Sections | 40+ Topics | 400+ Commands

Preface & Introduction

3

1

Essential System Operations

Core Linux Concepts & Commands

4

2

User & Group Management

Users, Groups, Permissions & PAM

5

3

Storage Management

Disks, Filesystems, LVM & RAID

6

4

Networking

IP, DNS, Firewall & SSH

7

5

Service Management & Security

systemd Units, Cron, Logging & Hardening

8

6

Shell Scripting & Automation

Bash Scripting, Variables & Text Processing

9

7

System Monitoring & Troubleshooting

Performance, Diagnostics & Debugging Tools

10

8

Advanced Topics & Virtualization

Containers, KVM, Kernel Modules & Advanced Config

11

9

Kernel & Boot Management

GRUB, initramfs, Boot Process & Recovery

12

10

Security & Exam Preparation

Hardening, Audit, Encryption & LFCS Exam Tips

13

10 Sections

40+ Topics

400+ Commands

2025 Edition

PREFACE & INTRODUCTION

Welcome to the LFCS Study Notes & Reference Guide — your concise, hands-on companion for Linux Foundation Certified Sysadmin exam preparation.

This guide distills the most important Linux commands, concepts, and key points across all 10 LFCS topic areas. Each section presents commands in a clear reference table format alongside highlighted key points to reinforce understanding.

How to use this guide: Use it as a quick-reference companion alongside your practical lab work. Focus on understanding the *why* behind each command, not just memorizing syntax. The LFCS exam is performance-based — hands-on practice is essential.

Best of luck in your LFCS journey! — *Akhilesh Chaudhari*

Section 1: Essential System Operations

Core Linux Concepts & Commands

Linux Kernel & System Info

<code>uname -r</code>	Show current kernel version
<code>uname -a</code>	Show all system info — kernel, hostname, arch
<code>cat /proc/version</code>	Detailed kernel version & build info
<code>cat /proc/cpuinfo</code>	CPU details — cores, model, speed
<code>cat /proc/meminfo</code>	RAM details — total, free, available
<code>lscpu</code>	CPU architecture summary
<code>lshw -short</code>	Hardware inventory summary
<code>hostnamectl</code>	Show hostname, OS, kernel info
<code>hostnamectl set-hostname NAME</code>	Permanently change hostname

□ KEY POINTS

- MemAvailable in `/proc/meminfo` is the real indicator of free usable RAM
- LFCS certification validates basic to intermediate Linux sysadmin skills
- `/proc` is a virtual filesystem — data is generated dynamically by kernel

systemd & Service Management

<code>systemctl start/stop/restart svc</code>	Control service state immediately
<code>systemctl enable/disable svc</code>	Enable/disable at boot
<code>systemctl enable --now svc</code>	Enable AND start in one command
<code>systemctl status svc</code>	Show service status & recent logs
<code>systemctl is-enabled svc</code>	Check if service starts at boot
<code>systemctl list-units --type=service</code>	List all service units
<code>systemctl daemon-reload</code>	Reload after unit file changes
<code>systemctl get-default</code>	Show default target (runlevel)
<code>systemctl set-default multi-user.target</code>	Set default to text mode

□ KEY POINTS

- systemd is PID 1 — first process started by kernel
- Targets replace runlevels: `multi-user.target` = runlevel 3, `graphical.target` = runlevel 5
- Always run `daemon-reload` after editing unit files in `/etc/systemd/system/`

journalctl — Log Management

<code>journalctl -u nginx</code>	Logs for specific service
<code>journalctl -f</code>	Follow logs in real time
<code>journalctl -p err</code>	Show only error messages
<code>journalctl -b</code>	Logs from current boot

<code>journalctl -b -1</code>	Logs from previous boot
<code>journalctl --since today</code>	Today's logs only
<code>journalctl --since '2025-01-01'</code>	Logs from a specific date
<code>journalctl --list-boots</code>	List all recorded boot sessions

□ KEY POINTS

- `journalctl` reads from `systemd`'s binary journal — much more structured than `syslog`
- Priority levels: `emerg(0)` `alert(1)` `crit(2)` `err(3)` `warning(4)` `notice(5)` `info(6)` `debug(7)`

Process Management

<code>ps aux</code>	List all processes with CPU/MEM %
<code>ps aux --sort=-%cpu head</code>	Top CPU consuming processes
<code>top / htop</code>	Interactive real-time process monitor
<code>kill PID</code>	Send <code>SIGTERM</code> (graceful stop)
<code>kill -9 PID</code>	Send <code>SIGKILL</code> (force terminate)
<code>kill -HUP PID</code>	Send <code>SIGHUP</code> (reload config)
<code>nice -n 10 cmd</code>	Start with lower priority (+10)
<code>renice -n 5 -p PID</code>	Change priority of running process
<code>nohup cmd &</code>	Run immune to hangup, background

□ KEY POINTS

- Nice range: -20 (highest priority) to 19 (lowest). Only root can set negative values
- `SIGKILL (9)` cannot be caught or ignored — it always terminates the process
- Zombie processes: completed but parent hasn't called `wait()` — kill parent to reap

Section 2: User & Group Management

Users, Groups, Permissions & PAM

User Management Commands

<code>useradd -m -s /bin/bash -G sudo USER</code>	Create user with home, shell, group
<code>usermod -aG docker USER</code>	Add user to group (append, not replace)
<code>usermod -L USER / -U USER</code>	Lock / Unlock user account
<code>userdel -r USER</code>	Delete user + home directory + mail
<code>passwd USER</code>	Set or change user password
<code>passwd -l USER / -u USER</code>	Lock / Unlock via passwd command
<code>id USER</code>	Show UID, GID, all groups
<code>groups USER</code>	Show group memberships
<code>whoami</code>	Show current username

□ KEY POINTS

- `/etc/passwd` fields: username:x:UID:GID:GECOS:home:shell – 'x' means hash in `/etc/shadow`
- `/etc/shadow` fields: username:hash:last_changed:min:max:warn:inactive:expire
- UIDs 0=root, 1-999=system accounts, 1000+=regular users

Password Aging — chage

<code>chage -l USER</code>	List current password aging settings
<code>chage -M 90 USER</code>	Set max password age to 90 days
<code>chage -m 7 USER</code>	Set minimum password age to 7 days
<code>chage -W 14 USER</code>	Warn 14 days before expiry
<code>chage -d 0 USER</code>	Force password change at next login
<code>chage -E 2025-12-31 USER</code>	Set account expiry date
<code>faillock --user USER</code>	Show failed login attempts
<code>faillock --reset --user USER</code>	Reset/unlock failed login counter

□ KEY POINTS

- Configure defaults in `/etc/login.defs`: `PASS_MAX_DAYS`, `PASS_MIN_DAYS`, `PASS_WARN_AGE`
- `pam_pwquality` enforces complexity – configure `/etc/security/pwquality.conf`
- `pam_faillock` handles lockout – configure `/etc/security/faillock.conf`

Groups & Permissions

<code>groupadd -g 1500 devs</code>	Create group with specific GID
<code>groupmod -n newname old</code>	Rename a group
<code>groupdel GROUP</code>	Delete a group
<code>newgrp GROUP</code>	Switch active primary group for session
<code>chmod 755 file</code>	Set permissions (rwxr-xr-x)

<code>chmod u+x,g-w file</code>	Symbolic permission change
<code>chown user:group file</code>	Change owner and group
<code>chown -R user:group dir/</code>	Recursive ownership change
<code>umask 022</code>	Default creation mask (files=644, dirs=755)

□ KEY POINTS

- Special bits: SUID=4xxx (run as owner), SGID=2xxx (inherit group), Sticky=1xxx (only owner can delete)
- Find SUID files: `find / -perm -4000 -ls 2>/dev/null`
- umask subtracts from 666 (files) and 777 (dirs) — umask 027 gives files 640, dirs 750

ACLs — Fine-Grained Permissions

<code>setfacl -m u:alice:rw file</code>	Grant alice read-write access
<code>setfacl -m g:devs:r file</code>	Grant devs group read access
<code>setfacl -x u:alice file</code>	Remove alice's ACL entry
<code>setfacl -b file</code>	Remove ALL ACL entries from file
<code>setfacl -R -m u:alice:rw dir/</code>	Recursive ACL set
<code>getfacl file</code>	Display all ACL entries

□ KEY POINTS

- ACLs require filesystem support — ext4 and XFS support them by default
- Files with ACL show '+' in `ls -l` output: `-rw-rw-r--+`
- Mask entry restricts max effective permissions for ACL entries

Section 3: Storage Management

Disks, Filesystems, LVM & RAID

Disk & Partition Tools

<code>lsblk -f</code>	List block devices with filesystem & UUID
<code>fdisk /dev/sdb</code>	Interactive MBR partition editor
<code>gdisk /dev/sdb</code>	Interactive GPT partition editor
<code>parted /dev/sdb print</code>	Show GPT/MBR partition info
<code>blkid /dev/sdb1</code>	Show UUID & filesystem type
<code>mkfs.ext4 /dev/sdb1</code>	Format partition with ext4
<code>mkfs.xfs /dev/sdb1</code>	Format partition with XFS
<code>tune2fs -L LABEL /dev/sdb1</code>	Set volume label on ext4
<code>tune2fs -m 1 /dev/sdb1</code>	Reduce reserved blocks to 1%

□ KEY POINTS

- MBR: max 4 primary partitions, max 2TB. GPT: up to 128 partitions, supports >2TB
- Always use `UUID=` in `/etc/fstab` instead of `/dev/sdX` — device names can change at boot
- NEVER format without confirming device with `lsblk` first — data is permanently lost

Mount & fstab

<code>mount /dev/sdb1 /mnt/data</code>	Mount a partition
<code>mount -o ro /dev/sdb1 /mnt</code>	Mount read-only
<code>mount -a</code>	Mount all entries in <code>/etc/fstab</code>
<code>umount /mnt/data</code>	Unmount a filesystem
<code>umount -l /mnt/data</code>	Lazy unmount if busy
<code>lsof /mnt/data</code>	Find processes using a mount point
<code>findmnt --verify</code>	Validate <code>/etc/fstab</code> syntax
<code>df -h</code>	Show disk usage of all mounts
<code>df -i</code>	Show inode usage (important!)

□ KEY POINTS

- `/etc/fstab` fields: device mountpoint fstype options dump fsck-order
- Common options: defaults, ro, noexec, nosuid, noatime, nofail, _netdev
- noatime mount option improves performance — skips access time updates on reads

LVM — Logical Volume Manager

<code>pvcreate /dev/sdb /dev/sdc</code>	Initialize physical volumes
<code>vgcreate datavg /dev/sdb /dev/sdc</code>	Create volume group
<code>lvcreate -L 50G -n datalv datavg</code>	Create 50GB logical volume
<code>lvextend -L +10G /dev/datavg/datalv</code>	Extend LV by 10GB

<code>resize2fs /dev/datavg/data1v</code>	Resize ext4 filesystem online
<code>xfs_growfs /mnt/data</code>	Grow XFS filesystem online
<code>lvresize -r -L +10G /dev/datavg/data1v</code>	Extend LV + resize fs together
<code>lvcreate -s -n snap01 -L 5G /dev/datavg/data1v</code>	Create snapshot
<code>pvs / vgs / lvs</code>	Show PV / VG / LV summary

□ KEY POINTS

- LVM layers: Physical Volumes (PV) □ Volume Groups (VG) □ Logical Volumes (LV)
- LV paths: `/dev/VGname/LVname` or `/dev/mapper/VGname-LVname`
- XFS cannot be shrunk — only grown. ext4 can be shrunk but must be unmounted first

RAID with mdadm

<code>mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sdb /dev/sdc</code>	Create RAID 1
<code>cat /proc/mdstat</code>	Monitor RAID sync status
<code>mdadm --detail /dev/md0</code>	Show RAID array details
<code>mdadm /dev/md0 --add /dev/sdd</code>	Add spare disk to array
<code>mdadm --detail --scan >> /etc/mdadm/mdadm.conf</code>	Save RAID config

□ KEY POINTS

- RAID 0: Striping — speed, no redundancy, min 2 disks
- RAID 1: Mirroring — full redundancy, can lose 1 disk, min 2 disks
- RAID 5: Distributed parity — can lose 1 disk, min 3 disks, usable = N-1 disks
- RAID 6: Dual parity — can lose 2 disks, min 4 disks, usable = N-2 disks

Section 4: Networking

IP, DNS, Firewall & SSH

ip Command — Network Configuration

ip addr show / ip a	Show all IP addresses
ip link show	Show network interfaces
ip route show / ip r	Show routing table
ip addr add 192.168.1.10/24 dev eth0	Assign IP to interface
ip link set eth0 up/down	Enable or disable interface
ip route add default via 192.168.1.1	Add default gateway
ip neigh show	Show ARP table
ip -s link show eth0	Show interface statistics

□ KEY POINTS

- ip command changes are NOT persistent — use NetworkManager or config files
- nmcli connection modify ... persists changes to /etc/NetworkManager/system-connections/
- On Ubuntu 18.04+: use Netplan YAML files in /etc/netplan/ — apply with netplan apply

DNS & Name Resolution

dig domain.com	Full DNS lookup with details
dig @8.8.8.8 domain.com	Query specific DNS server
dig -x 1.2.3.4	Reverse DNS lookup
dig +trace domain.com	Trace full resolution path
nslookup domain.com	Simple DNS lookup
resolvectl status	Show systemd-resolved config
getent hosts hostname	Test full resolution chain

□ KEY POINTS

- Resolution order defined in /etc/nsswitch.conf — 'hosts: files dns' checks /etc/hosts first
- /etc/resolv.conf sets nameservers — on systemd systems managed by systemd-resolved
- /etc/hosts provides static overrides — takes effect immediately without restart

Firewall — firewalld & iptables

firewall-cmd --state	Check firewalld status
firewall-cmd --zone=public --add-service=http --permanent	Allow HTTP permanently
firewall-cmd --zone=public --add-port=8080/tcp --permanent	Allow port 8080
firewall-cmd --reload	Apply permanent rules
firewall-cmd --list-all	Show all zone rules
iptables -L -n -v	List all iptables rules

iptables -A INPUT -p tcp --dport 22 -j ACCEPT	Allow SSH
iptables-save > /etc/iptables/rules.v4	Persist iptables rules
ufw enable / ufw status verbose	Enable UFW / show rules

□ KEY POINTS

- firewalld uses zones — default zone is 'public'. Use --permanent to persist rules
- iptables rules are NOT persistent without saving — install iptables-persistent
- nftables is the modern replacement: nft list ruleset shows all rules

SSH — Secure Shell

ssh-keygen -t ed25519	Generate ED25519 key pair (recommended)
ssh-copy-id user@server	Copy public key to server
ssh -i ~/.ssh/id_ed25519 user@host	Connect with specific key
ssh -J bastion user@internal	ProxyJump through bastion host
ssh -L 8080:host:80 user@server	Local port forwarding
scp -r dir/ user@host:/path/	Recursive secure copy
rsync -avz --delete src/ user@host:/dst/	Efficient sync over SSH

□ KEY POINTS

- Key hardening in /etc/ssh/sshd_config: PermitRootLogin no, PasswordAuthentication no
- ED25519 keys are preferred over RSA — smaller, faster, equally secure
- ProxyJump (-J) is safer than agent forwarding (-A) for bastion hosts
- Always test config with 'sshd -t' before restarting SSH service

Section 5: Service Management & Security

systemd Units, Cron, Logging & Hardening

systemd Unit File Structure

[Unit] Description=, After=, Wants=	Unit metadata & dependencies
[Service] Type=simple/forking/oneshot	Service type definition
[Service] ExecStart=/path/to/binary	Main start command
[Service] Restart=on-failure, RestartSec=5	Auto-restart on failure
[Service] User=appuser, Group=appgroup	Run as specific user
[Service] EnvironmentFile=/etc/app/env	Load env vars from file
[Service] LimitNOFILE=65535	Set open files limit
[Install] WantedBy=multi-user.target	Enable target

□ KEY POINTS

- Unit files location: /etc/systemd/system/ (custom) or /lib/systemd/system/ (package)
- Type=simple: process stays foreground. Type=forking: daemonizes. Type=notify: sends ready signal
- Always run systemctl daemon-reload after creating or modifying unit files

Cron & Scheduled Tasks

crontab -e	Edit current user's crontab
crontab -l	List current cron jobs
crontab -r	Remove all cron jobs
0 2 * * 0 /backup.sh	Run at 2 AM every Sunday
*/15 * * * * /check.sh	Run every 15 minutes
@reboot /startup.sh	Run once at boot
@daily /rotate.sh	Run once daily
systemctl list-timers	List all systemd timers

□ KEY POINTS

- Cron format: min(0-59) hour(0-23) day(1-31) month(1-12) weekday(0-7) command
- Always use full paths in cron — cron has minimal PATH environment
- Redirect output: '* * * * * /cmd >> /var/log/cmd.log 2>&1' to capture errors
- systemd timers: better logging, dependency support, missed-run catch-up (Persistent=true)

SELinux & AppArmor

getenforce	Show SELinux mode: Enforcing/Permissive/Disabled
setenforce 0	Temporarily set Permissive mode
sestatus	Detailed SELinux status
ausearch -m AVC -ts recent	Find recent SELinux denials
audit2allow -w -a	Explain denials in human language

restorecon -Rv /path	Restore correct file SELinux contexts
aa-status	Show AppArmor profiles & modes
aa-enforce /etc/apparmor.d/profile	Put profile in enforce mode

□ KEY POINTS

- SELinux modes: Enforcing (blocks+logs), Permissive (logs only), Disabled
- Permanent SELinux config: /etc/selinux/config — set SELINUX=enforcing
- AppArmor uses path-based policies (Ubuntu/SUSE). SELinux uses label-based (RHEL/CentOS)

Log Management

journalctl -u svc -f	Follow service logs live
journalctl -p err -b	Errors from current boot
grep 'Failed' /var/log/auth.log	Find failed SSH attempts
grep 'sudo' /var/log/auth.log	Find sudo usage
logrotate -d /etc/logrotate.d/app	Dry-run test logrotate config
logrotate -f /etc/logrotate.conf	Force log rotation now
logger -p auth.info 'test'	Send test message to syslog

□ KEY POINTS

- Log files: /var/log/auth.log (Debian) or /var/log/secure (RHEL) for authentication
- rsyslog config: /etc/rsyslog.conf — routes by facility.severity to log files
- logrotate options: daily/weekly, rotate N, compress, delaycompress, copytruncate

Section 6: Shell Scripting & Automation

Bash Scripting, Variables & Text Processing

Bash Script Essentials

<code>#!/bin/bash</code>	Shebang — specify interpreter
<code>set -euo pipefail</code>	Exit on error, undefined vars, pipe failures
<code>chmod +x script.sh && ./script.sh</code>	Make executable and run
<code>bash -x script.sh</code>	Debug mode — print each command
<code>bash -n script.sh</code>	Syntax check without running
<code>shellcheck script.sh</code>	Static analysis for bugs & pitfalls
<code>trap 'cleanup' EXIT INT TERM</code>	Cleanup on exit or signal
<code>local var=value</code>	Scope variable inside function

□ KEY POINTS

- Always use `set -euo pipefail` at top of scripts for safety
- Use `#!/usr/bin/env bash` for portability across different systems
- `shellcheck` catches common mistakes — integrate into CI/CD pipeline

Variables & String Operations

<code>VAR=value / echo \$VAR</code>	Assign and read variable
<code>\${VAR:-default}</code>	Use default if VAR is unset/empty
<code>\${VAR:?error msg}</code>	Exit with error if VAR unset
<code>\${#VAR}</code>	Length of string
<code>\${VAR:2:5}</code>	Substring from pos 2, length 5
<code>\${VAR//old/new}</code>	Replace ALL occurrences
<code>\${VAR^^} / \${VAR,,}</code>	Uppercase / lowercase
<code>\$(command)</code>	Command substitution (preferred over backticks)
<code>\$(5 * 3)</code>	Arithmetic expansion — integers only

□ KEY POINTS

- Always quote variables: `"$VAR"` — prevents word splitting and glob expansion
- Single quotes: literal (no expansion). Double quotes: allows `$VAR` and `$(cmd)`
- For floating point math use: `echo 'scale=2; 5/3' | bc`

Text Processing — grep, sed, awk

<code>grep -inrv 'pattern' dir/</code>	Case-insensitive, line numbers, recursive, invert
<code>grep -A3 -B3 'pattern' file</code>	3 lines after and before match
<code>grep -E 'pat1 pat2' file</code>	Extended regex — match either pattern
<code>sed 's/old/new/g' file</code>	Replace all occurrences globally
<code>sed -i.bak 's/old/new/g' file</code>	In-place edit with backup

sed -n '/pattern/p' file	Print only matching lines
sed '/pattern/d' file	Delete matching lines
awk -F: '{print \$1,\$3}' /etc/passwd	Print fields 1 & 3 (colon delimiter)
awk '{sum+=\$2} END{print sum}' file	Sum a column

□ KEY POINTS

- grep -P enables Perl regex: grep -P '\d{3}-\d{4}' (phone numbers)
- sed address ranges: sed '5,10d' (delete lines 5-10), sed '/start/,/end/d'
- awk NR (record number) and NF (number of fields) are very useful built-in vars

Loops, Conditions & Functions

for f in *.txt; do echo \$f; done	Loop over files
for ((i=0; i<5; i++)); do echo \$i; done	C-style for loop
while IFS= read -r line; do echo \$line; done < file	Safe line reading
if [-f file]; then ... fi	POSIX condition check
[[\$str =~ ^[0-9]+\$]]	Bash regex match
[-f] -d -e -r -w -x -z -n	File & string test operators
func() { local v=1; echo \$v; }	Define function with local var
result=\$(get_value)	Capture function output

□ KEY POINTS

- Use [[]] over [] in bash — no word splitting, supports regex with =~
- Always quote file paths in tests: if [[-f "\$file"]]
- While loop with read is the safest way to process files line by line

Section 7: System Monitoring & Troubleshooting

Performance, Diagnostics & Debugging Tools

Performance Monitoring Tools

top / htop	Interactive CPU, memory, process monitor
vmstat 1 5	CPU, memory, swap, I/O stats every 1 sec
iostat -x 1	Disk I/O stats — %util, await, r/s, w/s
iotop -o	Per-process disk I/O usage
free -h	RAM and swap usage summary
sar -u 1 5	Historical CPU stats (sysstat)
sar -r 1	Historical memory stats
sar -n DEV 1	Network interface stats
dstat	Combined vmstat+iostat+ifstat output

□ KEY POINTS

- vmstat key columns: r=runqueue, b=blocked, si/so=swap in/out, wa=iowait
- iostat %util near 100% = disk saturation. await = I/O latency in milliseconds
- Enable sysstat for historical data: `systemctl enable --now sysstat`

Load Average & Memory

uptime	System uptime and 1/5/15 min load average
nproc	Number of available CPU cores
free -h	Show RAM: total, used, free, available
cat /proc/meminfo grep MemAvailable	True usable RAM
dmesg grep -i 'out of memory'	Find OOM killer events
smem -s rss -r head	Per-process real memory usage
vmstat -s	Memory stats summary

□ KEY POINTS

- Load average > number of CPU cores = overloaded system
- MemAvailable (not MemFree) is the real indicator of usable RAM
- OOM killer score: `cat /proc/PID/oom_score` — higher = more likely killed

Disk & File Diagnostics

du -sh /path	Total size of directory (human readable)
du -ah /var sort -rh head -20	Top 20 largest files/dirs
df -h	Filesystem disk space usage
df -i	Inode usage — important when 'disk full' but space exists
lsof /mnt/data	Processes keeping a mount busy
lsof -p PID	All files opened by a process

smartctl -H /dev/sda	S.M.A.R.T. disk health check
fsck /dev/sdb1	Check & repair filesystem (unmounted only)

□ KEY POINTS

- 'No space left on device' with free disk space = inode exhaustion — check df -i
- S.M.A.R.T. attributes: Reallocated_Sector_Ct non-zero = disk failing
- Run fsck only on unmounted filesystems — use live media for root partition

Network Diagnostics

ss -tuln	All listening TCP/UDP sockets (no DNS lookup)
ss -tp	TCP connections with process names
ping -c 4 host	Test ICMP connectivity
traceroute / tracepath host	Show network hops to destination
mtr google.com	Combined ping+traceroute live view
tcpdump -i eth0 port 80	Capture HTTP traffic on eth0
tcpdump -w capture.pcap	Save capture for Wireshark analysis
curl -v https://host	Verbose HTTP request with headers

□ KEY POINTS

- Troubleshoot order: link up? □ IP assigned? □ ping localhost □ ping gateway □ ping 8.8.8.8 □ DNS
- ss replaces netstat — faster, more info. 'ss -tuln' = 'netstat -tuln'
- tcpdump requires root or CAP_NET_ADMIN capability

Section 8: Advanced Topics & Virtualization

Containers, KVM, Kernel Modules & Advanced Config

Linux Namespaces & cgroups

lsns	List all namespaces on the system
unshare --pid --fork bash	Create new PID namespace
nsenter -t PID --net --pid bash	Enter process's namespaces
cat /sys/fs/cgroup/memory/.../memory.limit_in_bytes	cgroup memory limit
systemd-cgtop	Real-time cgroup resource usage
systemctl set-property svc CPUQuota=50%	Set CPU limit on service
systemctl set-property svc MemoryMax=512M	Set memory limit

□ KEY POINTS

- 7 namespace types: PID, Network, Mount, UTS, IPC, User, Cgroup
- Containers = namespaces (isolation) + cgroups (resource limits) + overlayfs
- cgroups v2 unified hierarchy at /sys/fs/cgroup/ – used by modern systemd

KVM Virtual Machine Management

grep -E '(vmx svm)' /proc/cpuinfo	Check CPU virtualization support
virsh list --all	List all VMs (running + stopped)
virsh start/shutdown/reboot VM	Control VM state
virsh console VM	Connect to VM console
virsh snapshot-create-as VM snap01	Create VM snapshot
virt-install --name vm1 --ram 2048 --disk size=20 --cdrom img.iso	Create new VM
virsh dominfo VM	Show VM configuration details

□ KEY POINTS

- KVM requires: CPU with vmx (Intel) or svm (AMD) support + enabled in BIOS
- VM config stored as XML in /etc/libvirt/qemu/. Edit with virsh edit VM
- virt-manager provides GUI management on top of libvirt/KVM

Kernel Modules

lsmod	List all currently loaded modules
modinfo modulename	Show module info & parameters
modprobe modulename	Load module with dependencies
modprobe -r modulename	Remove module
depmod -a	Rebuild module dependency database
cat /etc/modules-load.d/custom.conf	Modules loaded at boot (systemd)
cat /etc/modprobe.d/blacklist.conf	Blacklisted modules

▣ KEY POINTS

- modprobe is preferred over insmod – resolves and loads dependencies automatically
- Blacklist modules in /etc/modprobe.d/*.conf: 'blacklist nouveau'
- /sys/module/ contains parameters of loaded modules – writable for runtime tuning

System Tuning – sysctl

sysctl -a	Show all kernel parameters
sysctl -w vm.swappiness=10	Temporarily change a parameter
sysctl -p	Reload /etc/sysctl.conf
echo 'vm.swappiness=10' >> /etc/sysctl.d/99-custom.conf	Persist parameter
net.ipv4.ip_forward=1	Enable IP forwarding (for routing/NAT)
net.core.somaxconn=65535	Increase listen backlog (web servers)
vm.overcommit_memory=1	Allow memory overcommit (Redis/VMs)
kernel.panic=30	Auto-reboot 30s after kernel panic

▣ KEY POINTS

- sysctl changes are temporary – persist in /etc/sysctl.d/*.conf files
- Web server tuning: net.core.somaxconn, net.ipv4.tcp_max_syn_backlog
- Database tuning: vm.swappiness=10, vm.dirty_ratio=5

Section 9: Kernel & Boot Management

GRUB, initramfs, Boot Process & Recovery

Linux Boot Process

Step 1: Power ON	CPU executes BIOS/UEFI firmware
Step 2: POST	Hardware power-on self test
Step 3: Bootloader	BIOS reads MBR / UEFI reads ESP □ loads GRUB2
Step 4: Kernel load	GRUB loads vmlinuz + initramfs into RAM
Step 5: Kernel init	Kernel decompresses, initializes hardware
Step 6: Init system	Kernel starts /sbin/init (PID 1) = systemd
Step 7: Targets	systemd starts services & login manager

□ KEY POINTS

- initramfs = temporary RAM filesystem with drivers to mount the real root filesystem
- Rebuild initramfs: update-initramfs -u (Debian) or dracut -f (RHEL)
- UEFI: stores bootloaders in EFI System Partition (ESP) at /boot/efi — FAT32

GRUB2 Configuration

/etc/default/grub	Main GRUB settings file — edit this
GRUB_TIMEOUT=5	Boot menu display timeout in seconds
GRUB_CMDLINE_LINUX_DEFAULT	Kernel parameters for normal boot
update-grub	Regenerate grub.cfg (Debian/Ubuntu)
grub2-mkconfig -o /boot/grub2/grub.cfg	Regenerate grub.cfg (RHEL)
grub-install /dev/sda	Reinstall GRUB to MBR
efibootmgr -v	List UEFI boot entries

□ KEY POINTS

- NEVER edit /boot/grub/grub.cfg directly — it is auto-generated
- Edit /etc/default/grub, then run update-grub to apply changes
- Kernel params for recovery: 'single' or 't' = single-user, 'rd.break' = initramfs shell

System Recovery

Edit GRUB entry □ append 'init=/bin/bash'	Emergency root shell
mount -o remount,rw /	Remount root as read-write in recovery
passwd root	Reset root password in recovery
touch /.autorelabel	Re-label SELinux after recovery (RHEL)
grub-install /dev/sda	Reinstall GRUB from live media chroot
update-grub	Regenerate GRUB config in chroot
fsck /dev/sda1	Check/repair filesystem from live media

□ KEY POINTS

- Chroot rescue: mount /dev/sda1 /mnt □ bind mounts □ chroot /mnt □ fix □ reboot
- Bind mounts for chroot: mount --bind /dev /mnt/dev && /proc /mnt/proc && /sys /mnt/sys
- SysRq magic keys: Alt+SysRq+S (sync), U (remount ro), B (reboot) — REISUB sequence

Kernel Parameters & Modules at Boot

nomodeset	Disable kernel mode setting — fix graphics issues
quiet splash	Suppress boot messages, show splash screen
ro / rw	Mount root read-only / read-write at boot
ipv6.disable=1	Disable IPv6 entirely
mem=2G	Limit RAM usage to 2GB
crashkernel=auto	Reserve memory for kdump crash kernel
/etc/modules-load.d/*.conf	Modules to load at boot
/etc/modprobe.d/*.conf	Module options and blacklist

□ KEY POINTS

- Add kernel params in GRUB_CMDLINE_LINUX in /etc/default/grub, then update-grub
- sysctl params set at boot via /etc/sysctl.d/ files — applied by systemd-sysctl.service
- dracut --list-modules shows available initramfs modules on RHEL systems

Section 10: Security & Exam Preparation

Hardening, Audit, Encryption & LFCS Exam Tips

Security Hardening Checklist

Disable unused services	systemctl disable --now service
SSH: disable root login	PermitRootLogin no in sshd_config
SSH: keys only, no passwords	PasswordAuthentication no
Firewall: allow only needed ports	firewall-cmd or ufw rules
SELinux/AppArmor: enforcing mode	getenforce / aa-status
fail2ban: brute force protection	systemctl enable --now fail2ban
Auto security updates	unattended-upgrades (Debian) / dnf-automatic (RHEL)
File integrity monitoring	aide --check (AIDE) or Tripwire
Audit logging	systemctl enable --now auditd

□ KEY POINTS

- Defense in depth: multiple layers — firewall + SELinux + fail2ban + audit + updates
- Principle of least privilege: grant minimum permissions needed for the task
- Regular log review: `grep 'Failed' /var/log/auth.log && journalctl -p err -b`

Disk Encryption — LUKS

<code>cryptsetup luksFormat /dev/sdb1</code>	Initialize LUKS encryption
<code>cryptsetup open /dev/sdb1 mydata</code>	Open/decrypt to /dev/mapper/mydata
<code>mkfs.ext4 /dev/mapper/mydata</code>	Format the decrypted device
<code>mount /dev/mapper/mydata /mnt</code>	Mount and use
<code>cryptsetup luksAddKey /dev/sdb1</code>	Add additional passphrase/key
<code>cryptsetup luksDump /dev/sdb1</code>	Show LUKS header info
<code>cryptsetup close mydata</code>	Close/lock the encrypted device

□ KEY POINTS

- LUKS is the Linux standard for full disk/partition encryption
- For boot-time unlock: add entry to /etc/crypttab
- Backup LUKS header: `cryptsetup luksHeaderBackup /dev/sdb1 --header-backup-file header.bak`

auditd — Security Audit Framework

<code>systemctl enable --now auditd</code>	Enable audit daemon
<code>auditctl -w /etc/passwd -p wa -k passwd_watch</code>	Watch file for writes
<code>auditctl -a always,exit -F arch=b64 -S execve -k exec_log</code>	Log all executions
<code>ausearch -k passwd_watch</code>	Search audit log by key
<code>ausearch -m AVC -ts recent</code>	Find recent SELinux denials

aureport -au	Authentication summary report
aureport -x --summary	Executed commands summary

▣ KEY POINTS

- Audit rules persist in /etc/audit/rules.d/*.rules — loaded at boot
- auditd required for PCI-DSS, HIPAA, SOX compliance
- ausearch -ui UID finds all actions by a specific user ID

LFCS Exam Tips & Key Topics

Essential Ops (25%)	File management, man pages, basic commands
User/Group Mgmt (15%)	useradd, usermod, sudo, PAM, chage
Networking (20%)	ip, nmcli, firewall, SSH, DNS, ss
Storage (25%)	LVM, filesystems, mount, RAID, fstab, quotas
Service Mgmt (15%)	systemctl, unit files, cron, logging
Practical exam — 2 hours	Hands-on tasks on real Linux systems
No multiple choice	Must actually perform tasks to pass

▣ KEY POINTS

- LFCS is performance-based — practice hands-on, not just theory
- Study with a real Linux VM: Ubuntu or CentOS recommended
- Key skill: combining tools with pipes, scripts, and troubleshooting methodology
- Always read man pages during exam — man command is your best friend